CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
An Autonomous Institute | Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in
Approved by | Affiliated to | UGC Autonomous | 14 Programs Accredited by NBA | Grade A++ in NAAC | All India Ranking 151-200 Band nirf | Certifications Quality ISO 21001 : 2018 Green ISO 14001 : 2015 Energy ISO 50001 : 2018
COMMITTED TO RESEARCH, INNOVATION AND EDUCATION 48 years

Craft Cataloger: Classifying Indian Handicrafts by Fine-Tuning a Pre-trained CNN

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A)

(Affiliated to Osmania University)

CASE STUDY REPORT

Submitted By:

160124748116 – Leela Sai Charan Putti

160124748131 – Suneet Reddy

CASE STUDY GUIDE

Panigrahi Srikanth, Assistant Professor

Department of CSE Artificial Intelligence and Machine Learning

CBIT, Hyderabad

Certificate

This is to certify that the project report entitled **Craft Cataloger: Classifying Indian Handicrafts by Fine-Tuning a Pre-trained CNN** has been carried out by the following students during the academic year 2025-2026 under my guidance and supervision.

160124748116 – Leela Sai Charan Putti
160124748131 – Suneet Reddy

Project Guide:

Panigrahi Srikanth
Assistant Professor
Department of CSE AI&ML
CBIT, Hyderabad

Acknowledgement

We express our sincere gratitude to everyone who supported this project. We thank our guide, **Panigrahi Srikanth** sir, for his continuous encouragement, insightful suggestions, and expert guidance throughout this work. His knowledge of deep learning and computer vision was instrumental in shaping the fine-tuning approach used for recognising Indian handicrafts.

We thank the Head of the Department, **Dr. Y. Ramadevi** mam, for providing the necessary resources and an enabling environment to complete this project.
Finally, we are grateful to our family and friends for their unwavering support and motivation.

Abstract

Artisans and small sellers often find it difficult to list handicraft products online, because writing a category, title and description for every item takes time and domain knowledge. This report presents **Craft Cataloger**, a reduced-scope deep learning prototype of the ShilpSetu AI idea. It predicts the craft style of a product from a single photograph and turns the prediction into a ready-to-use catalog card.

Using the public Kaggle *Indian Handicraft Products* dataset, a balanced subset of eight craft classes was prepared, with 400 images per class split into 2,240 training, 480 validation and 480 test images. An ImageNet-pretrained ResNet18 was adapted in two stages: first only a new classification layer was trained, and then the last residual block was fine-tuned at a lower learning rate. The final model reaches **86.5% accuracy** (415 of 480 images) and a macro F1-score of 0.86 on the held-out test set. Oxidised metal craft is recognised perfectly, while most errors occur among visually similar block-printed textiles.

A Gradio interface and an optional Whisper voice-note module demonstrate how the classifier can support automatic cataloging. The entire project was implemented in Google Colab using free resources.

List of Figures

6.1 Training loss (left) and validation accuracy (right) over all 14 epochs. Stage 2 (fine-tuning) starts at index 6 on the horizontal axis. 13
6.2 Confusion matrix of the final model on the test set. Rows are true classes and columns are predicted classes. 16
6.3 Gradio demo output for an uploaded product photo, showing the class probabilities and the generated catalog card. 18

List of Tables

3.1 Software and libraries used 6
3.2 Fields in the dataset 7
3.3 Selected classes and number of cleaned listings available 7

4.1 Two-stage training configuration 10

5.1 Training hyperparameters 12

6.1 Training loss and validation accuracy per epoch 14
6.2 Accuracy after each stage of training 14
6.3 Per-class results on the test set (60 images per class) 15

8.1 Catalog card text for each class 22

Table of Contents

S.No	Title	Page No.
1.	**1. Introduction**	1
	1.1 Background and Motivation	1
	1.2 The Role of Deep Learning in Craft Recognition	1
	1.3 Key Challenges in Handicraft Image Classification	1
	1.4 Objectives and Report Structure	2
2.	**2. Literature Review and Theoretical Study**	3
	2.1 Evolution of Image Classification Models	3
	2.1.1 Convolutional Neural Network Architectures	3
	2.1.2 Efficient Networks and Vision Transformers	3
	2.2 Fine-Tuning of Pre-trained Models	3
	2.2.1 Feature Extraction versus Fine-Tuning	3
	2.2.2 Dataset Augmentation	4
	2.3 Theoretical Background of ResNet18	4
	2.4 Supporting Tools: Whisper and Gradio	4
	2.5 Scope and Motivation of This Work	4
3.	**3. System Requirements and Specifications**	6
	3.1 Hardware and Software Requirements	6
	3.1.1 Hardware Specifications	6
	3.1.2 Software and Frameworks	6
	3.2 Dataset Description and Characteristics	7
	3.3 Evaluation Metrics	8
4.	**4. Methodology and System Design**	9
	4.1 Methodology Workflow	9
	4.2 Proposed Architecture: ResNet18 with a New Classifier	9
	4.2.1 Pretrained Backbone	9
	4.2.2 Replacement Classification Head	10
	4.3 Two-Stage Training Strategy	10
	4.4 Catalog Card Generation	10
5.	**5. Implementation Details and Experimental Setup**	11
	5.1 Implementation Environment	11
	5.2 Training Protocol	11
	5.2.1 Data Pipeline and Augmentation	11
	5.2.2 Loss Function	11

S.No	Title	Page No.
	5.2.3 Optimization and Hyperparameters	12
	5.3 Demo Application	12
6.	**6. Results and Performance Analysis**	13
	6.1 Overview	13
	6.2 Training Behaviour	13
	6.3 Test-Set Performance	14
	6.3.1 Overall Results	14
	6.3.2 Per-Class Performance	15
	6.3.3 Confusion Matrix	16
	6.4 Interpretation of Results	16
	6.4.1 Why Fine-Tuning Helps	16
	6.4.2 Strong and Weak Classes	16
	6.4.3 Overfitting	17
	6.5 Demo Application Output	17
	6.6 Summary of Findings	18
7.	**7. Conclusion and Future Scope**	19
	7.1 Conclusion	19
	7.2 Limitations	19
	7.3 Unified Research and Deployment Roadmap	20
8.	**8. Appendix I: Model Details and Preprocessing**	21
	8.1 Label Extraction and Cleaning Procedure	21
	8.2 Catalog Card Templates	21
	8.3 Voice-Note Module	22
9.	**9. References**	23

1. Introduction

1.1 Background and Motivation

India has one of the richest handicraft traditions in the world, with regional textile and metalwork styles such as bandhani tie-dye, ajrakh block printing, pochampally ikat weaving and kalamkari. Many artisans sell through intermediaries or online marketplaces, but listing a product online requires a clear category, title and description for every item. Writing this information is slow, needs language and domain knowledge, and is a real barrier for small producers.

This project is inspired by *ShilpSetu AI*, an idea for an AI-driven market-linkage and smart-cataloging application for marginalised artisans. The full idea has several parts, such as buyer matching and voice input. For the Deep Learning Lab, only the **smart-cataloging component** is implemented: given a photograph of a product, the system predicts the craft style and produces a catalog card.

1.2 The Role of Deep Learning in Craft Recognition

Recognising a craft style from a photograph is an image classification problem. Craft styles differ in texture, colour palette, motif and surface finish, and these are the kinds of patterns that convolutional neural networks (CNNs) learn well from data. Large networks pretrained on ImageNet [1] already encode general visual features such as edges, textures and shapes. Reusing these features by fine-tuning a pre-trained model makes it possible to obtain a useful classifier from a few hundred images per class instead of millions.

1.3 Key Challenges in Handicraft Image Classification

- **Visually similar styles.** Several block-printing traditions use related motifs, dyes and colour schemes, so the differences between classes are subtle.

- **Noisy and multi-valued labels.** The dataset comes from web product listings. The art form of a product is stored as a list of tags, and some tags (for example *handmade*) describe the production method rather than a visual style.
- **Limited data and compute.** The project had to run on free cloud resources, which rules out training large models from scratch.
- **Class imbalance.** Some art forms have thousands of listings while others have only a few hundred, so the data must be balanced before training.

1.4 Objectives and Report Structure

The objectives of this work are:

1. To build a clean, balanced image dataset of Indian craft styles from the public Kaggle dataset.
2. To train and fine-tune a pretrained CNN to classify eight craft styles.
3. To evaluate the model on unseen data using accuracy, precision, recall, F1-score and a confusion matrix.
4. To demonstrate a working prototype that converts a prediction into a catalog card.

The rest of the report is organised as follows. Chapter 2 reviews the relevant theory. Chapter 3 lists the requirements, the dataset and the evaluation metrics. Chapter 4 describes the methodology and model design, and Chapter 5 gives the implementation details. Chapter 6 presents and analyses the results, and Chapter 7 concludes with limitations and future scope. The appendix contains supporting details.

2. Literature Review and Theoretical Study

2.1 Evolution of Image Classification Models

2.1.1 Convolutional Neural Network Architectures

Modern image classification was transformed when AlexNet won the ImageNet challenge using a deep CNN trained on GPUs. VGG networks showed that stacking many small 3×3 convolutions gives strong accuracy, but very deep plain networks become hard to optimise. ResNet [2] solved this by adding skip connections, which let networks with tens or hundreds of layers train reliably and made residual networks a standard backbone for fine-tuning pre-trained models.

2.1.2 Efficient Networks and Vision Transformers

Later work focused on efficiency and new architectures. MobileNetV2 and EfficientNet achieve good accuracy with fewer parameters and less computation, which suits phones and low-cost servers. Vision Transformers split an image into patches and process them with self-attention, and they perform very well when pretrained on large datasets. For a small dataset and a short lab project, a compact pretrained CNN is the more practical choice.

2.2 Fine-Tuning of Pre-trained Models

2.2.1 Feature Extraction versus Fine-Tuning

A pre-trained network carries knowledge learned on a large source task (here, ImageNet classification) over to a smaller target task. It is well known that early layers of a CNN learn general features (edges, colours, textures) while later layers become more task-specific. This suggests a two-stage recipe that this project follows:

1. **Feature extraction:** freeze the pretrained backbone and train only a new classification layer.

2. **Fine-tuning:** unfreeze the last block of the backbone and train it with a small learning rate, so that the high-level features adapt to the new classes without destroying the pretrained weights.

2.2.2 Dataset Augmentation

Dataset augmentation creates modified copies of training images, such as flips and small colour changes, to reduce overfitting. It is especially useful with only a few hundred images per class. In this project, random horizontal flips and mild colour jitter are applied to the training images only.

2.3 Theoretical Background of ResNet18

A residual block learns a correction to its input instead of a full transformation. If x is the input to the block and $\mathcal{F}(x)$ is the output of its convolutional layers, the block output is

$$y = \mathcal{F}(x) + x. \tag{2.1}$$

The identity shortcut lets gradients flow directly to earlier layers, which makes deep networks easier to train [2]. ResNet18 contains an initial convolution and pooling layer, four stages of residual blocks (called `layer1` to `layer4`), global average pooling, and one fully connected layer. It has about 11.7 million parameters in its original 1000-class form, which is small enough to fine-tune quickly on a single GPU.

2.4 Supporting Tools: Whisper and Gradio

Whisper [3] is a general-purpose speech recognition model trained on a large amount of multilingual audio. It is used here as an optional module that converts an artisan's spoken description into text. Gradio [4] is a Python library that wraps a model in a web interface with a few lines of code, which makes it suitable for demonstrating the classifier without building a backend.

2.5 Scope and Motivation of This Work

This project does not try to advance the state of the art in image classification. Its aim is practical: to show that a small pipeline built from free tools (a public dataset, a pretrained CNN

and a free cloud GPU) can recognise several craft styles well enough to automate the first step of cataloging, and to analyse honestly where it succeeds and where it fails.

3. System Requirements and Specifications

3.1 Hardware and Software Requirements

3.1.1 Hardware Specifications

All experiments were run on Google Colab, so no special local hardware was needed. The training runtime used a Colab GPU, which is required to fine-tune the network in reasonable time. The user's own computer only needs a modern web browser and an internet connection.

3.1.2 Software and Frameworks

Table 3.1 lists the main software used.

Table 3.1: Software and libraries used

Library / Tool	**Purpose**
Python 3 (Google Colab)	Programming language and runtime
PyTorch, torchvision [5]	Model definition, pretrained ResNet18, training, image transforms
pandas	Loading and cleaning the CSV files
scikit-learn [6]	Stratified splitting, classification report, confusion matrix
Pillow	Reading images
matplotlib	Plotting curves and the confusion matrix
kagglehub	Downloading the dataset
Gradio [4]	Demo web interface
OpenAI Whisper [3]	Optional speech-to-text module

3.2 Dataset Description and Characteristics

The dataset is *Indian Handicraft Products* [7], published on Kaggle under the MIT licence. It contains three CSV files with a combined 37,273 product listings, and a folder of about 37,300 product images (roughly 1.17 GB). Each listing has the five fields shown in Table 3.2.

Table 3.2: Fields in the dataset

Field	Description
`product_title`	Title of the product listing
`product_description`	Text description of the product
`price`	Listed price
`image_paths`	Relative path(s) of the product image(s), stored as a list in text form
`artform`	Art form tag(s), stored as a list in text form (for example `['ajrakh block printing', 'natural dyed']`)

Label preparation. Because `artform` is a list, the first tag was used as the class label and the first image as the model input. Tags that name a production method rather than a visual style (*fabart*, *handmade* and *plain solid*) were removed, along with rows whose image file was missing. After cleaning, the eight most frequent art forms were selected. Table 3.3 shows how many cleaned listings each had; each class was then capped at 400 images so that the classes are balanced.

Table 3.3: Selected classes and number of cleaned listings available

Class	Listings available	Images used
Ajrakh block printing	1,744	400
Sanganeri block printing	1,604	400
Bandhani tie dye	1,503	400
Oxidised metal craft	1,475	400
Pochampally ikat weaving	1,177	400
Kalamkari block printing	1,082	400
Jacquard weaving	908	400
Batik block printing	877	400
Total		**3,200**

The 3,200 selected images were split in a stratified way into training, validation and test sets in the ratio 70/15/15 with a fixed random seed of 42. This gave **2,240 training, 480 validation**

and 480 test images, i.e. 280, 60 and 60 images per class respectively.

3.3 Evaluation Metrics

Let TP_c, FP_c and FN_c be the true positives, false positives and false negatives of class c, and N the total number of test images. The following metrics are used.

Accuracy is the fraction of correctly classified images:

$$\text{Accuracy} = \frac{\sum_c TP_c}{N}. \tag{3.1}$$

Precision, recall and F1-score are computed for each class:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2\,P_c R_c}{P_c + R_c}. \tag{3.2}$$

Precision tells how many images predicted as class c are correct, and recall tells how many true images of class c were found. The **macro average** is the unweighted mean of the per-class values, and it treats all classes equally. Finally, the **confusion matrix** counts how many images of each true class were predicted as each class, which shows exactly which classes are mixed up.

4. Methodology and System Design

4.1 Methodology Workflow

The pipeline has the following steps.

1. **Load data:** download the dataset and combine the three CSV files.
2. **Clean labels:** extract the first art form tag and first image, drop generic tags and rows with missing images.
3. **Select and balance classes:** keep the top eight classes and at most 400 images per class.
4. **Split:** create stratified 70/15/15 training, validation and test sets.
5. **Preprocess:** resize images to 224×224 and normalise them with ImageNet statistics; apply augmentation to training images only.
6. **Stage 1 training:** freeze the pretrained backbone and train a new classification layer.
7. **Stage 2 training:** unfreeze `layer4` and fine-tune it together with the classifier.
8. **Evaluate:** compute metrics on the test set and plot the confusion matrix.
9. **Demonstrate:** wrap the trained model in a Gradio interface that outputs a catalog card.

4.2 Proposed Architecture: ResNet18 with a New Classifier

4.2.1 Pretrained Backbone

The backbone is ResNet18 with weights pretrained on ImageNet. It was chosen because it is small, fast to fine-tune on a free GPU, and has a well-understood architecture [2]. The convolutional layers, from the first convolution to `layer4`, produce a 512-dimensional feature vector for each image after global average pooling.

4.2.2 Replacement Classification Head

The original 1000-class fully connected layer is replaced by a new linear layer that maps the 512 features to eight outputs, one per craft class. A softmax over these outputs gives the class probabilities that the demo shows as confidence. Replacing the head leaves the model with about 11.2 million parameters in total.

4.3 Two-Stage Training Strategy

Table 4.1 summarises the two stages. In Stage 1 only the new head is trained, which is fast and safe because the pretrained features are not changed. In Stage 2 the last residual block, which holds the most task-specific features, is unfrozen. It uses a learning rate ten times smaller than the head so that the pretrained weights are adjusted gently.

Table 4.1: Two-stage training configuration

Stage	**Trainable layers**	**Learning rate**	**Epochs**	**Trainable params**
1. Feature extraction	Final layer only	10^{-3}	6	$\approx$ 4 thousand
2. Fine-tuning	`layer4` + final layer	10^{-4} / 10^{-3}	8	$\approx$ 8.4 million

4.4 Catalog Card Generation

After classification, the predicted class is mapped to a catalog entry containing a craft name, a category, a short description, tags and the model's confidence. The descriptions are written templates (listed in Appendix I), not text generated by a language model. This keeps the prototype simple and predictable, and the confidence value shows the user how certain the model is.

5. Implementation Details and Experimental Setup

5.1 Implementation Environment

The whole project is a single Google Colab notebook written in Python with PyTorch. The dataset is downloaded with `kagglehub`, the model is trained on the Colab GPU, and the trained weights are saved as a checkpoint file (`craft_resnet18.pt`) together with the list of class names.

5.2 Training Protocol

5.2.1 Data Pipeline and Augmentation

A custom PyTorch `Dataset` reads each image with Pillow, converts it to RGB and applies the transforms below. Data is loaded in batches of 32 using two worker processes.

- **Training:** resize to 224×224, random horizontal flip, colour jitter (brightness and contrast 0.2), conversion to tensor and ImageNet normalisation.
- **Validation and test:** resize to 224×224, conversion to tensor and ImageNet normalisation only, with no random changes.

5.2.2 Loss Function

The model is trained with the cross-entropy loss, which is standard for multi-class classification:

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^{B} \log p_{i,y_i}, \tag{5.1}$$

where B is the batch size and p_{i,y_i} is the probability the model assigns to the true class y_i of image i.

5.2.3 Optimization and Hyperparameters

Both stages use the Adam optimiser [8]. Table 5.1 lists the complete configuration.

Table 5.1: Training hyperparameters

Setting	**Value**
Backbone	ResNet18 (ImageNet weights)
Input size	224×224 RGB
Optimiser	Adam
Learning rate, Stage 1 (head)	10^{-3}
Learning rate, Stage 2 (`layer4` / head)	10^{-4} / 10^{-3}
Epochs	6 (Stage 1) + 8 (Stage 2) = 14
Batch size	32
Loss	Cross-entropy
Data split	70 / 15 / 15, stratified, seed 42
Model used for testing	Weights after the final epoch

5.3 Demo Application

The trained model is wrapped in a Gradio interface. The user uploads a product photo, and the interface returns the top three predicted classes with probabilities and a formatted catalog card. A second, optional interface uses the Whisper *base* model to transcribe a short voice note, which represents how an artisan could describe a product by speaking instead of typing.

6. Results and Performance Analysis

6.1 Overview

This chapter reports how the model trained, how it performs on the 480 unseen test images, and which classes it confuses. The validation set was used to monitor training, and the test set was used only once, for the final evaluation of the model after the last epoch.

6.2 Training Behaviour

Figure 6.1 plots the training loss and validation accuracy over the epochs, and Table 6.1 lists the values after every epoch. In Stage 1 (epochs 1–6) the loss falls from 1.731 to 0.869 and validation accuracy rises from 54.6% to 72.1%. As soon as Stage 2 begins, validation accuracy jumps to 80.8% in the first epoch and reaches its peak of 86.3% at epoch 8. The training loss falls to 0.046 by the end.

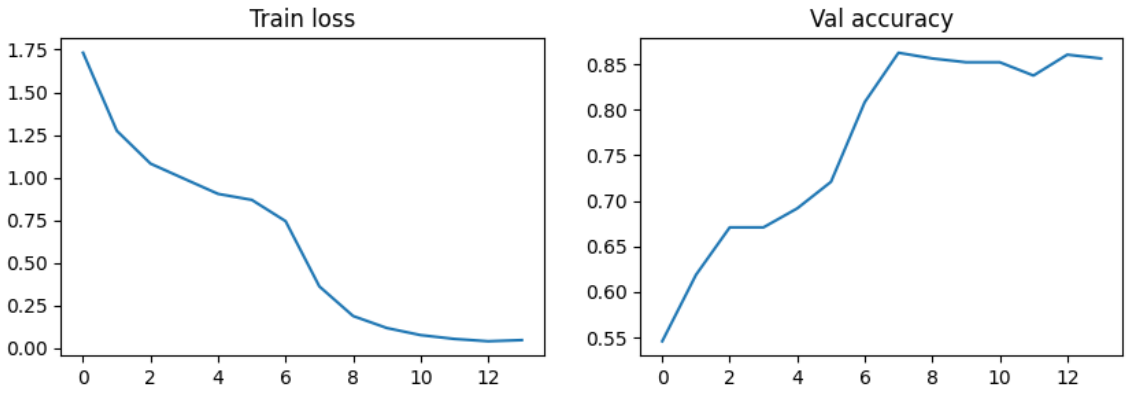


Figure 6.1: Training loss (left) and validation accuracy (right) over all 14 epochs. Stage 2 (fine-tuning) starts at index 6 on the horizontal axis.

Table 6.1: Training loss and validation accuracy per epoch

Epoch	Stage	Train loss	Val. accuracy
1	1	1.731	54.6%
2	1	1.274	61.9%
3	1	1.081	67.1%
4	1	0.992	67.1%
5	1	0.904	69.2%
6	1	0.869	72.1%
7	2	0.744	80.8%
8	2	0.361	86.3%
9	2	0.188	85.6%
10	2	0.117	85.2%
11	2	0.076	85.2%
12	2	0.053	83.8%
13	2	0.040	86.0%
14	2	0.046	85.6%

6.3 Test-Set Performance

6.3.1 Overall Results

On the 480 test images the model classified **415 correctly, giving a test accuracy of 86.5%**. The macro-averaged precision, recall and F1-score are 0.87, 0.86 and 0.86 respectively; they are almost equal because every class has the same number (60) of test images. Table 6.2 compares the stages.

Table 6.2: Accuracy after each stage of training

Model state	Accuracy
After Stage 1 (head only), validation	72.1%
Stage 2, best epoch (epoch 8), validation	86.3%
Stage 2, final epoch (epoch 14), validation	85.6%
Final model, test set	**86.5%**

6.3.2 Per-Class Performance

Table 6.3 gives precision, recall and F1-score for every class, computed from the confusion matrix in Figure 6.2.

Table 6.3: Per-class results on the test set (60 images per class)

Class	**Precision**	**Recall**	**F1-score**
Ajrakh block printing	0.79	0.80	0.79
Bandhani tie dye	0.95	0.95	0.95
Batik block printing	0.92	0.75	0.83
Jacquard weaving	0.92	0.80	0.86
Kalamkari block printing	0.80	0.98	0.88
Oxidised metal craft	0.97	1.00	0.98
Pochampally ikat weaving	0.84	0.90	0.87
Sanganeri block printing	0.76	0.73	0.75
Macro average	**0.87**	**0.86**	**0.86**

6.3.3 Confusion Matrix

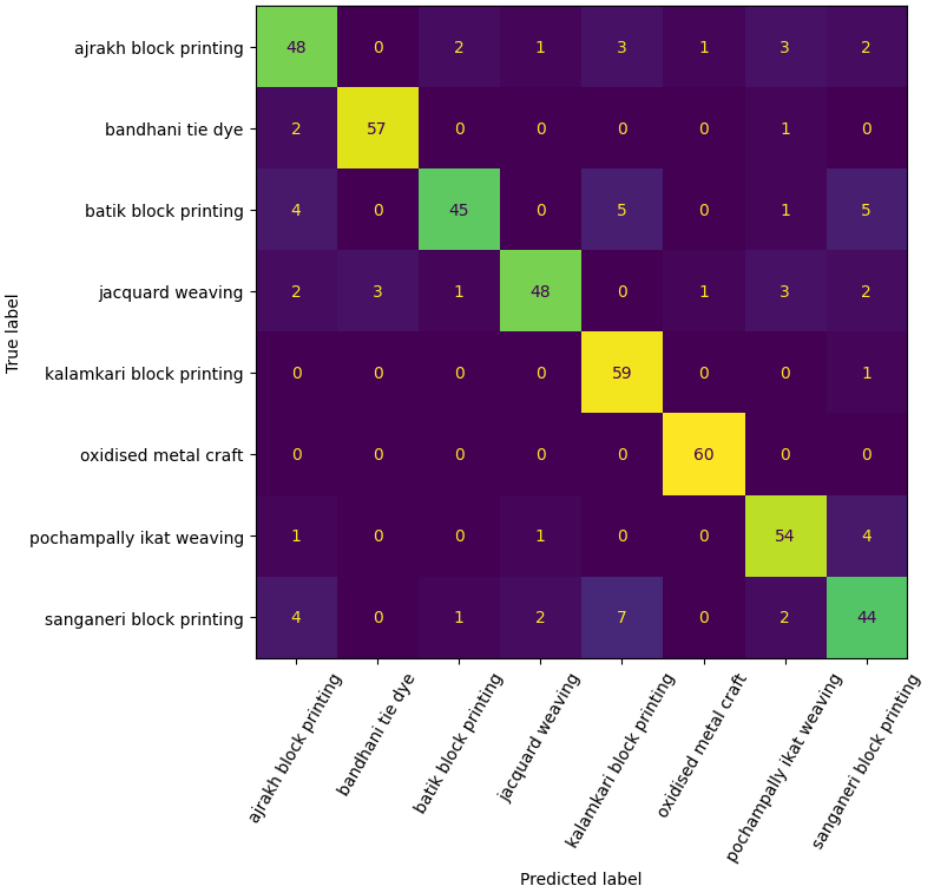


Figure 6.2: Confusion matrix of the final model on the test set. Rows are true classes and columns are predicted classes.

6.4 Interpretation of Results

6.4.1 Why Fine-Tuning Helps

Training only the new head reached 72.1% validation accuracy, so the frozen ImageNet features already carry useful information, but they are tuned for everyday objects rather than fabric prints. Unfreezing `layer4` let the high-level features adapt to textile patterns, which raised validation accuracy by about 9 percentage points in a single epoch and by about 14 points at the peak.

6.4.2 Strong and Weak Classes

Oxidised metal craft is recognised perfectly (all 60 images correct, recall 1.00). It is the only metal product among the eight classes, so its appearance (metallic surface and dark finish)

differs strongly from the six textile classes and the model rarely mistakes another class for it. Bandhani tie dye (recall 0.95) also has a distinctive dotted pattern.

The weakest class is sanganeri block printing (precision 0.76, recall 0.73), followed by batik (recall 0.75). The single largest confusion is sanganeri predicted as kalamkari (7 images), and batik images were split between kalamkari (5) and sanganeri (5). Kalamkari is over-predicted: 74 images were predicted as kalamkari while only 59 of them truly were (precision 0.80, recall 0.98). These block-printed styles share motifs, dye colours and cloth textures, and at 224×224 resolution the fine differences are difficult to see.

6.4.3 Overfitting

By the end of Stage 2 the training loss is close to zero, while validation accuracy stays around 84–86% and fluctuates slightly between epochs. This pattern suggests mild overfitting: the model fits the 2,240 training images very closely but gains little on unseen images. Because 480 test images are used, one image changes accuracy by about 0.2 percentage points and the standard error of the 86.5% estimate is about 1.6 points, so differences of one or two points between epochs or runs should not be over-interpreted.

6.5 Demo Application Output

For an uploaded photo, the Gradio interface displays the three most probable classes with their probabilities and a catalog card containing the craft name, category, description, tags and a confidence percentage. This confirms that the trained model can be used end to end: a seller uploads one photo and receives a draft listing that can be edited. The voice-note module separately returns the text transcript of a short audio clip.

Figure 6.3 shows the interface on a test photo of a red silk saree with a gold border. The model predicts *batik block printing* with 100% confidence. The saree appears to be woven rather than wax-resist printed, so this prediction looks wrong, and it also shows two limitations of the prototype: the softmax confidence can be very high for a wrong answer, and the model must choose one of its eight classes even for products that may not belong to any of them.

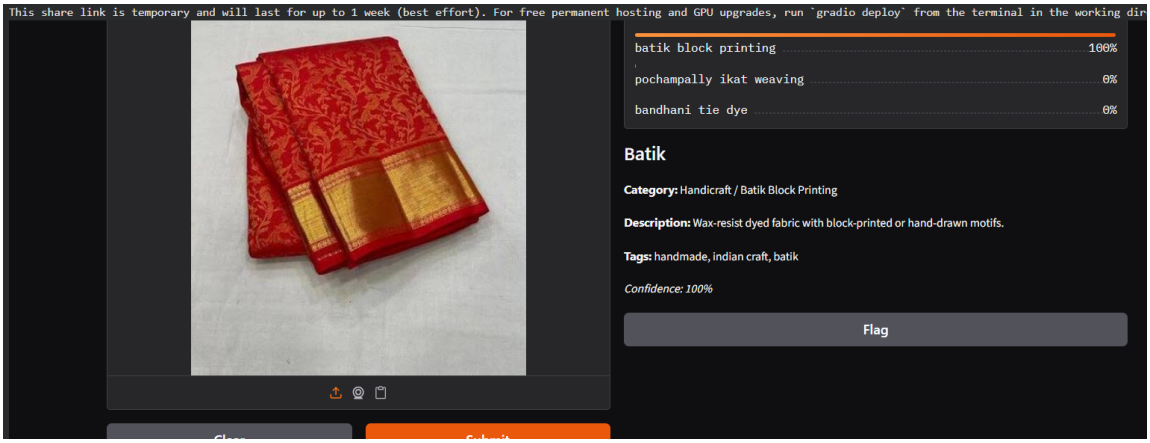


Figure 6.3: Gradio demo output for an uploaded product photo, showing the class probabilities and the generated catalog card.

6.6 Summary of Findings

- Fine-tuning a pre-trained ResNet18 gives 86.5% test accuracy on eight craft classes using only 400 images per class.
- Fine-tuning the last residual block was essential; it improved validation accuracy from 72.1% to a peak of 86.3%.
- Oxidised metal craft and bandhani are recognised most reliably, while block-printed textiles (sanganeri, batik, ajrakh, kalamkari) are confused with one another.
- The model shows mild overfitting, and the results come from a single train/validation/test split.

7. Conclusion and Future Scope

7.1 Conclusion

This project implemented the smart-cataloging part of the ShilpSetu AI idea as a deep learning lab case study. A noisy, web-collected dataset of Indian handicraft listings was cleaned into a balanced set of eight craft classes, and a pretrained ResNet18 was adapted with a two-stage fine-tuning strategy. The final model classifies unseen product photos with 86.5% accuracy and a macro F1-score of 0.86, and a Gradio demo turns each prediction into a catalog card. The work shows that a useful craft recogniser can be built entirely with free tools and a modest amount of data, and the error analysis identifies visually similar block-printing styles as the main remaining difficulty.

7.2 Limitations

While the results are encouraging, several limitations must be acknowledged:

- **Label quality.** Labels come from web listings, and only the first art form tag of each product was used, so some images may carry incomplete or inconsistent labels.
- **Small scope.** Only 8 classes and 400 images per class were used, so the model cannot recognise other crafts, and it will assign one of the eight classes even to an unrelated photo.
- **Single split.** The 86.5% figure comes from one train/validation/test split and one training run. No cross-validation or repeated runs were performed.
- **Final-epoch weights.** The test evaluation used the weights from the last epoch, not the epoch with the best validation accuracy, and no early stopping was applied.
- **Overconfident predictions.** The softmax confidence is not calibrated, and the demo can show 100% confidence for a wrong class (Figure 6.3).

- **Template-based descriptions.** Catalog descriptions are fixed templates per class and are not generated from the specific product in the photo.
- **Image-only input.** The product title and description text in the dataset were not used by the model.

7.3 Unified Research and Deployment Roadmap

Short term (model improvements).

- Keep the checkpoint with the best validation accuracy and add early stopping.
- Try stronger backbones such as EfficientNet or MobileNetV2, and compare them with ResNet18 in a table.
- Add stronger augmentation, clean the labels, and evaluate with cross-validation.
- Use Grad-CAM to visualise which image regions drive each prediction and to check that the model looks at the fabric rather than the background.

Medium term (richer cataloging).

- Use the multi-tag `artform` field as a multi-label problem (for example style plus dye type).
- Combine image and text features (title and description) in a multimodal model.
- Generate product-specific descriptions with a language model instead of fixed templates.

Deployment (towards ShilpSetu AI).

- Package the model in a mobile application with a lightweight backend.
- Support voice input in regional languages through speech recognition [3].
- Add the market-linkage features of the original idea, such as buyer matching and price guidance.

8. Appendix I: Model Details and Preprocessing

8.1 Label Extraction and Cleaning Procedure

The `artform` and `image_paths` columns are stored as text that looks like a Python list. The following function converts such a value into its first element, which is used as the class label or image path.

```
import ast

def first(x):
    try:
        v = ast.literal_eval(x)
        return v[0] if v else None
    except Exception:
        return None

df['label'] = df['artform'].apply(first)
df['img']   = df['image_paths'].apply(first)
```

Rows with a missing label or image are dropped, labels are converted to lower case, image paths are joined to the dataset folder, rows whose file does not exist are removed, and the generic tags *fabart*, *handmade* and *plain solid* are filtered out.

8.2 Catalog Card Templates

Table 8.1 lists the craft name and description shown on the catalog card for each predicted class. Each card also shows the category (*Handicraft* followed by the class name), tags and the model's confidence.

Table 8.1: Catalog card text for each class

Class	Card name	Description shown
Ajrakh block printing	Ajrakh	Resist-printed with wooden blocks in indigo and madder red, with dense geometric patterns, traditionally from Sindh and Kutch.
Bandhani tie dye	Bandhani	Tie-dye textile where tiny knots are bound before dyeing to form dotted patterns, from Gujarat and Rajasthan.
Batik block printing	Batik	Wax-resist dyed fabric with block-printed or hand-drawn motifs.
Jacquard weaving	Jacquard Weave	Woven fabric with intricate patterns built into the weave using a jacquard loom.
Kalamkari block printing	Kalamkari	Block-printed or hand-painted cloth with natural dyes, known for detailed floral and mythological motifs.
Oxidised metal craft	Oxidised Metal Craft	Metalwork with a darkened antique finish that highlights engraved detail.
Pochampally ikat weaving	Pochampally Ikat	Handwoven fabric made from pre-dyed yarn, giving the signature geometric patterns of Telangana.
Sanganeri block printing	Sanganeri Print	Fine floral block prints on a light base, from Sanganer, Rajasthan.

8.3 Voice-Note Module

The optional voice-note module loads the Whisper *base* model and returns the transcript of an uploaded or recorded audio clip through a second Gradio interface. In this prototype the transcript is shown to the user but is not yet used to fill the catalog card automatically; combining the two is listed as future work.

9. References

[1] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[3] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 28 492–28 518.

[4] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, *Gradio: Hassle-free sharing and testing of machine learning models in the wild*, arXiv preprint arXiv:1906.02569, 2019.

[5] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] hrishikeshb80, *Indian handicraft products*, Kaggle dataset, MIT licence. [Online]. Available: `https://www.kaggle.com/datasets/hrishikeshb80/indian-handicraft-products`.

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.